Bioinformatics, 2026, pp. 1–12

Original Paper

Diseño y evaluación de una base de datos en MongoDB para elementos transponibles de PanTEon

Thomas Alejandro Toro Herrera,[1] Juan Sebastián Arias Rueda,[1] Juan Esteban Hernández Martínez[1] and Isabela Quintero Murillo[1]

[1]Ingeniería de Sistemas, Universidad Autónoma de Manizales, Manizales, Colombia

Resumen

Motivación: Las bases de elementos transponibles (TEs) se distribuyen como archivos planos, que dificultan las consultas entre especies y no incluyen los perfiles de k-mers que usan los clasificadores. Aquí se diseña y evalúa una base en MongoDB para los 239.946 TEs de 2.790 especies de PanTEon.

Resultados: El modelo usa cuatro colecciones (especies, TEs, secuencias y k-mers de $k = 1$ a 6), combina referencias con campos copiados y se carga con un pipeline reproducible en Python. Responde 17 consultas y genera bibliotecas FASTA por especie y por reino. Frente a embeber los k-mers en cada TE, guardarlos en una colección aparte ocupa el mismo espacio en disco y hace entre 2,5 y 4,5 veces más rápidas las consultas que no usan k-mers.

Disponibilidad e implementación: Código, notebooks y resultados en `https://github.com/ThomasToro/transposable-elements-db`.

Contacto: isaquinteromurillo1304@gmail.com

Información suplementaria: Notebooks con el código y el resultado de cada consulta en el repositorio.

Palabras clave elementos transponibles, MongoDB, bases de datos NoSQL, k-mers, PanTEon

1. Introducción

Los elementos transponibles (TEs) son secuencias de ADN capaces de moverse o copiarse dentro del genoma. Están presentes en casi todos los eucariotas y se clasifican de forma jerárquica en clase, orden y superfamilia (Wicker et al., 2007). Los TEs de clase I, o retrotransposones, se copian a través de un intermediario de ARN, e incluyen los órdenes LTR, LINE y SINE. Los de clase II, o transposones de ADN, se mueven sin ese intermediario, como los TIR y los Helitron. Esa misma jerarquía aparece en el encabezado de cada secuencia de los datos usados en este trabajo, con la forma `CLASE/ORDEN/SUPERFAMILIA`.

Almacenar y analizar estos datos no es trivial. El conjunto usado reúne 239.946 TEs de 2.790 especies de animales, plantas y hongos, y su información está repartida en tres fuentes: las secuencias en un archivo FASTA, la taxonomía de cada especie en un CSV de metadatos y los perfiles de k-mers, que se calcularon en este trabajo. Los k-mers multiplican el volumen: con k de 1 a 6 hay 1.439.676 perfiles, que ocupan unos 8 GB sin comprimir. Además, los datos reales tienen inconsistencias, como 31.600 identificadores con un formato distinto al resto y nombres de especie que no coinciden entre el FASTA y el CSV. Con archivos planos, preguntas sencillas como cuántos TEs de cada orden hay en cada reino obligan a recorrer y cruzar los archivos completos.

Estos datos provienen de PanTEon, un marco entre reinos para guiar el diseño de clasificadores de TEs (Orozco-Arias et al., 2026a). Su base de datos es pública en Zenodo, versión 1.6.1, con licencia CC BY 4.0 (Orozco-Arias et al., 2026b).

En este trabajo se diseña, implementa y evalúa una base de datos documental en MongoDB (MongoDB, Inc., 2026a) para los datos de PanTEon. Las contribuciones son tres: (1) un modelo de datos que integra secuencias, taxonomía y k-mers, con cada decisión justificada; (2) un pipeline reproducible en Python que limpia, valida y carga los datos; y (3) una evaluación experimental de dos formas de guardar los k-mers, en una colección aparte o embebidos en cada TE. El artículo se organiza así: la sección de trabajo relacionado revisa cómo organizan esta información otras bases de datos; Materiales y métodos describe los datos, el modelo, el pipeline y los experimentos; Resultados presenta las consultas, las figuras y el benchmark; y la Discusión analiza las ventajas y limitaciones del diseño.

Conceptos básicos.

Para seguir el artículo bastan estas ideas:

- **k-mer.** Es cada pedazo de longitud k que se obtiene al recorrer una secuencia de a una letra. Por ejemplo, `ATGCA` tiene los 2-mers `AT`, `TG`, `GC` y `CA`. El *perfil de k-mers* de una secuencia cuenta cuántas veces aparece cada uno; con $k = 6$ hay hasta $4^6 = 4.096$ k-mers posibles.
- **Documento y colección.** MongoDB guarda los datos como documentos, parecidos a un JSON, y agrupa los documentos del

© The Author(s) 2026. Copyright and licence statements to be updated by the publisher during production.

mismo tipo en colecciones, que equivalen a las tablas de una base relacional.

- **Embebido o referencia.** Un dato relacionado se puede guardar *embebido*, dentro del mismo documento, o en otra colección, unido por una *referencia* (el `_id` del otro documento). Para juntar colecciones en una consulta se usa `$lookup`.
- **Índice.** Es una estructura que permite encontrar documentos sin recorrer toda la colección, como el índice de un libro. Con `explain` se ve si una consulta usó un índice (IXSCAN) o recorrió todos los documentos (COLLSCAN).
- **Validación de esquema.** Con `$jsonSchema`, MongoDB rechaza los documentos que no cumplen unas reglas, por ejemplo un campo obligatorio que falta.

2. Trabajo relacionado

Repbase es una de las bases de referencia de elementos repetitivos en eucariotas. Reúne secuencias consenso curadas, organizadas por grupo taxonómico y distribuidas como bibliotecas en archivos (Bao et al., 2015). En los datos de PanTEon, ocho identificadores de *Chlamydomonas reinhardtii* provienen de Repbase.

Dfam es un recurso comunitario de familias de TEs (Storer et al., 2021). Cada familia tiene una secuencia consenso, un modelo de perfil (HMM), una clasificación y una taxonomía asociada. Su uso principal es anotar genomas, para lo cual las familias se descargan como archivos HMM o FASTA. En PanTEon, 386 especies tienen como fuente Dfam y 31.592 identificadores conservan su nombre original de Dfam.

Para los genomas que no están en estas bases, las bibliotecas se construyen de forma automática. RepeatModeler2 descubre familias de TEs *de novo* en un genoma ensamblado (Flynn et al., 2020), y MCHelper cura automáticamente esas bibliotecas para reducir fragmentos, redundancia y clasificaciones erróneas (Orozco-Arias et al., 2024). PanTEon combina ambos orígenes: 2.404 especies con bibliotecas de RepeatModeler2 y MCHelper (RM2+MCH) y 386 de Dfam (Orozco-Arias et al., 2026a). Igual que las bases anteriores, se distribuye como archivos: un FASTA con las secuencias y un CSV con los metadatos.

Estos recursos están pensados para anotar genomas, no para consultas cruzadas sobre muchas especies, como comparar la composición de órdenes entre reinos o recuperar todas las secuencias de una superfamilia en una especie. Tampoco guardan los perfiles de k-mers junto a las secuencias, aunque son una entrada habitual de los clasificadores de TEs basados en aprendizaje automático, que son el uso que motiva PanTEon. Este trabajo integra secuencias, taxonomía y k-mers en una sola base consultable y evalúa experimentalmente cómo organizar los k-mers.

3. Materiales y métodos

3.1. Datos y problemas de calidad

Fuente: PanTEon Database v1.6.1, en Zenodo, licencia CC BY 4.0 (Orozco-Arias et al., 2026b).

El archivo FASTA cuenta con 239.946 registros en total, con encabezados de la forma `>ID#CLASE/ORDEN/SUPERFAMILIA @Especie`. Durante la exploración se encontraron 2 clases (Clase I: 194.774; Clase II: 45.172), 11 órdenes y 65 superfamilias (66 nombres, porque la de ARNt aparece escrita como `TRNA` y como `tRNA`), y todas las clasificaciones tienen 3 niveles. Los TEs pertenecen a 2.790 especies. Las secuencias tienen una longitud de 81 a 19.926 nucleótidos (mediana 4.259, media 4.017) y un contenido GC medio de 0,43 (de 0,11 a 0,80).

Tabla 1 Problemas encontrados en el FASTA y su decisión.

Problema	Casos	Decisión
Identificadores sin formato PDB + 8 dígitos	31.600 (Dfam 31.592, Repbase 8)	Se conservan porque son los nombres originales de las bibliotecas
Superfamilia igual al orden	21.544: 5.589 correctos, 10.319 clasificados solo hasta orden, 5.636 ambiguos	Se cargan tal cual y se documentan
Secuencias con minúsculas	208.354 (87 %)	Se conserva la secuencia; GC y k-mers se calculan pasando la secuencia a mayúsculas
Secuencias con N	54.837 (23 %; mediana 0,9 % de N)	Se conservan; se excluyen del GC y de las ventanas de k-mers
Especies que no cruzaban con el CSV	4 especies, 533 TEs	Para el cruce se quita el punto de subsp./sp.; en la base se guarda el nombre original

Tabla 2 Problemas encontrados en el CSV de metadatos y su decisión.

Problema	Filas	Decisión
Espacios al inicio o al final en la taxonomía	13	Se eliminan
Valores faltantes (`Phylum` 3, `Class` 49, `Order` 29, `Family` 76)	122	Se guardan como `null`
Rellenos de texto en `Order` (11 variantes, como *No trobat*)	26	Pasan a `null`
Comentarios en `Source 2` en lugar de fuentes	4	Pasan a `null`
`Species` que es un grupo taxonómico (como *Afrotheria*)	45	Se cargan con `is_species: false`, porque tienen TEs
Nombres no binomiales (subespecies, cepas, híbridos)	52	Se aceptan tal cual
Taxonomía contradictoria dentro de un género (como *Beta vulgaris* en dos familias)	18 (10 géneros)	Se cargan tal cual y se documentan
Especies duplicadas tras normalizar	0	Índice único en `species`

La Tabla 1 resume los problemas encontrados y su decisión.

En lo que no hubo problema: no se encontraron encabezados mal formados, identificadores duplicados, clasificaciones vacías o desconocidas (unknown, NA), caracteres distintos de A, C, G, T y N ni secuencias muy cortas para calcular 6-mers (la más corta tiene 81 nucleótidos).

Es importante aclarar que ningún dato original fue modificado; los problemas se resolvieron al calcular los atributos o al hacer el cruce, y están todos documentados.

El CSV de metadatos tiene 2.790 filas y 13 columnas, de las que se conservaron las 9 que pide el enunciado (`Kingdom`, `Phylum`, `Class`, `Order`, `Family`, `Species`, `Source`, `Source 2` y `Notes`). La Tabla 2 resume sus problemas y la decisión tomada con cada uno. Ningún valor se corrigió a mano: los rellenos y comentarios pasan a `null`, y las inconsistencias taxonómicas reales se cargan tal cual y se documentan.

El cruce se hace por especie, comparando los nombres normalizados con `normalize_species` (sin espacios sobrantes y en minúsculas). La primera versión dejó 4 especies sin pareja (533 TEs), porque el FASTA escribe *subsp.* y *sp.* con punto y el CSV sin él. Al eliminar

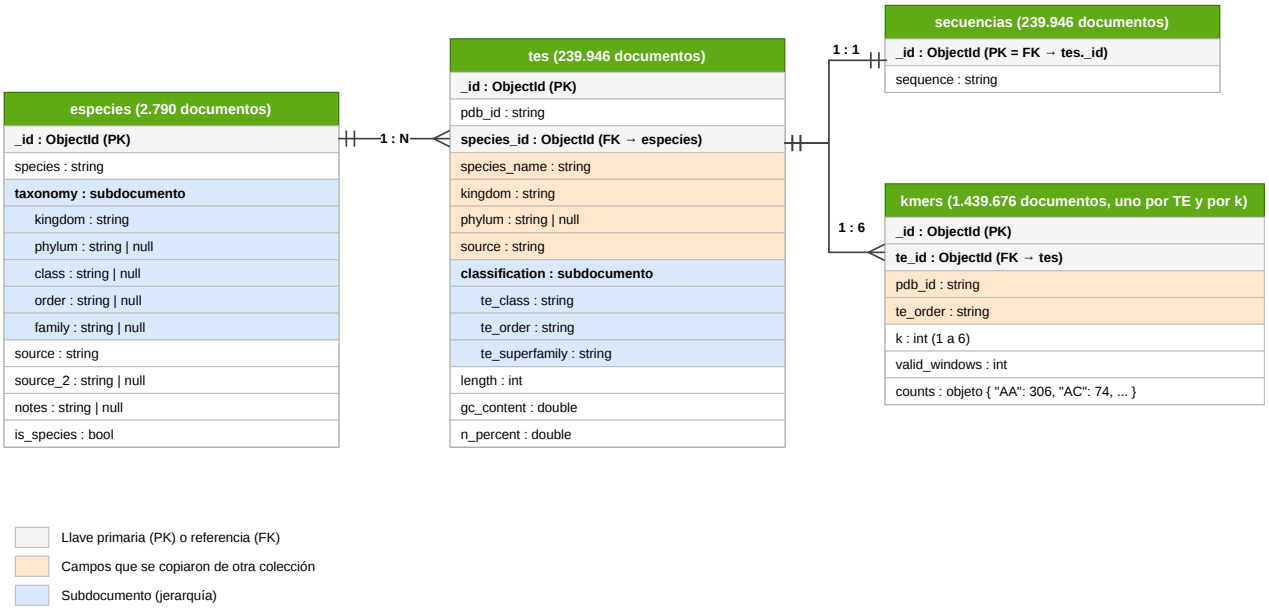


Figura 1 Modelo de datos en MongoDB: colecciones, campos y relaciones.

Texto alternativo: diagrama con cuatro tablas (especies, tes, secuencias y kmers) con sus campos y tipos, unidas por líneas que indican relaciones 1:N, 1:1 y 1:6. Los campos copiados están en naranja y los subdocumentos en azul.

también los puntos, las 2.790 especies cruzan en ambos sentidos y ningún TE queda sin `species_id`. Además, la columna `TE_count` del CSV, usada solo para validar, coincide con el número de TEs del FASTA en todas las especies.

3.2. Modelo de datos

La Figura 1 muestra el modelo de datos. La base tiene cuatro colecciones: `especies`, `tes`, `secuencias` y `kmers`. En `especies` hay un documento por especie (2.790), con su taxonomía completa agrupada en el subdocumento `taxonomy`. En `tes` hay un documento por elemento transponible (239.946), con su clasificación en el subdocumento `classification`, sus atributos calculados y una referencia a su especie (`species_id`). Además, `tes` copia algunos campos de la especie (en naranja en la figura) para no tener que hacer `$lookup` en las consultas más comunes. La secuencia de cada TE se guarda aparte, en `secuencias`, con el mismo `_id` que su TE (relación 1:1), y los k-mers se guardan en `kmers`, con seis documentos por TE, uno por cada valor de k (relación 1:6). Todas las relaciones usan el `_id` generado por MongoDB (ObjectId), y el identificador original de cada secuencia se guarda en el campo `pdb_id`.

3.3. Decisiones de diseño

La Tabla 3 resume las decisiones de diseño, con las alternativas descartadas y su motivo.

Modelos completos considerados.

Antes de fijar las decisiones, el equipo comparó tres modelos completos. El *normalizado* (taxonomía solo en `especies`, TEs con referencia y k-mers aparte) es el más limpio, pero depende de `$lookup` en todas las consultas por reino o phylum, que recorren unos 240.000 documentos. El *embebido* (taxonomía, secuencia y k-mers dentro de cada TE) es el más simple de consultar, pero cada TE pesa decenas de KB más y todas las consultas cargan con ese peso aunque no usen k-mers. Se eligió el *híbrido* de la Tabla 3, que equilibra velocidad de consulta y documentos livianos. Como los datos son una versión fija de PanTEon que no se actualiza, duplicar unos pocos campos no genera problemas de sincronización. Para el formato de los k-mers también se descartaron el arreglo de pares `{kmer, count}`, el más pesado y lento de cargar, y un binario comprimido, el más compacto pero imposible de consultar desde MongoDB, lo que incumpliría el requisito de que los perfiles se puedan consultar. La estrategia embebida se conservó como alternativa para el benchmark.

Tabla 3 Decisiones de diseño del modelo.

Decisión	Elección	Alternativas descartadas	Motivo
1. Relación entre TE-especie	Híbrida: `species_id` + copia de `species_name`, `kingdom`, `phylum` y `source`	Solo la referencia y la especie embebida completa.	Copiar solo algunos campos de la especie dentro del TE permite hacer las consultas por especie y por reino de forma más directa, entonces solo se copia lo que se necesita en la mayoría de los casos. Como los datos no cambian, la copia no genera problemas de actualización. Embeber todo repetiría la taxonomía en 239.946 documentos y solo referenciar obliga a hacer `$lookup` en casi todas las agregaciones.
2. Jerarquías	Subdocumentos: `taxonomy` en `especies` y `classification` en `tes`.	Campos planos; arreglo de ancestros; colección de clasificaciones	Con subdocumentos cada jerarquía queda agrupada y se evita confundir la clase y el orden de la especie con la clase y el orden del TE.
3. Secuencia	Colección aparte llamada `secuencias`, con el mismo `_id` que su TE	Dentro de `tes`	Se separó porque la secuencia es la parte más pesada de un TE (en promedio, 4.017 nucleótidos) y solo se usa para calcular los k-mers y generar las bibliotecas. Si estuviera dentro del documento del TE, todas las demás consultas tendrían que leerla aunque no la necesiten, y serían más lentas.
4. Ubicación de los k-mers	Colección `kmers`: un documento por TE y por k, con copia de `pdb_id` y `te_order`	Embebidos en el TE; un documento por TE con los 6 k; una colección por k	La consulta 15 y el heatmap usan un solo k a la vez, así cada una lee solo el k que necesita. El benchmark confirmó la elección: frente a guardar los k-mers dentro del TE, la carga, el espacio y las búsquedas por secuencia con índice fueron prácticamente iguales, pero las consultas que no usan k-mers fueron entre 2,5 y 4,5 veces más rápidas con la colección aparte.
5. Formato de los k-mers	Conteos en diccionario disperso + `valid_windows`	Arreglo denso, arreglo de pares y frecuencias	Se guardan los conteos enteros exactos junto con el total de ventanas, de modo que la frecuencia se puede calcular cuando se necesite; guardar frecuencias haría perder precisión. El diccionario solo incluye los k-mers presentes, así no guarda ceros (que abundan en k = 5 y 6), y es autodescriptivo: cada valor indica a qué k-mer corresponde.

Las decisiones se implementan con los siguientes índices, reglas de validación y controles de integridad.

Índices.

Se crean después de la carga, porque así la inserción es más rápida, y cada uno existe para una consulta concreta:

- `tes`: `{pdb_id}` único, para buscar una secuencia por su nombre (consulta 8) e impedir nombres repetidos; `{species_id}`, para el `$lookup` y los TEs de una especie; `{species_name, classification.te_superfamily}`, para la consulta 9 y la biblioteca por especie; y `{kingdom}`, para la biblioteca por reino.
- `kmers`: `{te_id, k}` único, para traer el perfil de un TE sin duplicados; `{pdb_id, k}`, para la consulta 15; y `{k, te_order}`, para el heatmap.
- `especies`: `{species}` único, para buscar por nombre e impedir especies repetidas.

Validación e integridad.

`especies`, `tes` y `kmers` tienen validación `$jsonSchema` estricta (`validationLevel: strict`, `validationAction: error`), es decir, MongoDB rechaza cualquier documento que no cumpla las reglas. La validación exige los campos obligatorios, restringe `kingdom` a los tres reinos, k a valores de 1 a 6 y `gc_content` y `n_percent` al intervalo $[0, 1]$. `secuencias` no tiene validador, porque guarda la secuencia tal como viene. Como MongoDB no garantiza la integridad referencial, el pipeline la protege con el orden de carga (`especies`, luego `tes` y `secuencias`, luego `kmers`), con verificaciones al final de cada etapa (todo `species_id` existe, `tes` y `secuencias` comparten los `_id` y cada TE tiene 6 perfiles) y con una guarda que impide recargar `tes` sin recalcular los k-mers, porque eso cambiaría los `_id`.

3.4. Pipeline en Python

El pipeline está escrito en Python 3.11 con pandas (McKinney, 2010), Biopython (Cock et al., 2009), numpy (Harris et al., 2020) y pymongo (MongoDB, Inc., 2026b). Se organiza en tres notebooks que se ejecutan en orden, con las funciones reutilizables en `src/` y pruebas automatizadas:

1. `01_pipeline_csv_especies`: limpia el CSV, carga `especies` con `bulk_write` de `ReplaceOne` con `upsert`, que reemplaza la especie si ya existe (así el notebook se puede volver a ejecutar sin cambiar los `_id`), crea el validador y el índice y genera el reporte del cruce.
2. `02_pipeline_fasta_tes`: lee el FASTA con Biopython, separa el encabezado por `#` y `@` sin suponer el prefijo PDB y valida caracteres, N y longitudes. Calcula tres atributos por TE: longitud, contenido GC ($G + C$ sobre $A + C + G + T$, sin contar N ni códigos IUPAC) y porcentaje de N. Une cada TE con su especie y carga `tes` y `secuencias` por lotes de 2.000 con `insert_many(ordered=False)`; los 479.892 documentos se cargaron en 6,6 s.
3. `03_pipeline_kmers`: calcula los perfiles de $k = 1$ a 6 y los carga en las dos estrategias del benchmark. La secuencia se pasa a mayúsculas y cada letra se convierte en un número (A = 0, C = 1, G = 2, T = 3). Así, cada ventana de k letras se convierte en un solo número en base 4, y numpy cuenta todas las ventanas de una vez con `np.bincount`, en lugar de recorrerlas una por una en Python. Toda ventana que contenga una N u otro carácter distinto de A, C, G y T se descarta, y `valid_windows` registra cuántas ventanas se contaron.

El cálculo de k-mers se validó contra una implementación en Python puro en 50 secuencias con N, códigos IUPAC y minúsculas. Con numpy, un solo proceso calcula unas 2.750 secuencias por segundo, y con 10 procesos (`multiprocessing`) unas 7.380 por segundo, medido en una muestra de 2.000 secuencias (`resultados/rendimiento_kmers.csv`). Eso equivale a unos 1,5 min para contar los k-mers de toda la base en un proceso; en la carga completa, el cálculo, que incluye también armar los documentos, tardó unos 125 s (Tabla 6). La reproducibilidad se apoya en el `README`

Tabla 4 Equipo en el que se ejecutaron todos los experimentos (`resultados/especificaciones_maquina_juan.md`).

Componente	Especificación
Procesador	Apple M5, 10 núcleos (4 de rendimiento y 6 de eficiencia)
Memoria RAM	24 GB
Disco	SSD interno (APFS), 460,4 GB
Sistema operativo	macOS 27.0.1
Base de datos	MongoDB 7.0.45, WiredTiger, caché de 11,5 GB
Software	Python 3.11.17, pymongo 4.19.0, numpy 2.4.6, pandas 3.1.0

(entorno con `environment.yml`), las pruebas (`pytest`) y los respaldos con `mongodump`.

3.5. Diseño de los experimentos

Para evaluar la decisión de almacenar los perfiles de k-mers en una colección aparte, se comparó esta estrategia con una alternativa en la que los perfiles se embeben en el documento de cada TE. Ambas contienen exactamente los mismos datos y solo difieren en su organización: en la estrategia principal, la colección `kmers` guarda un documento por TE y por valor de k (1.439.676 documentos) y se relaciona con `tes` mediante `te_id`; en la alternativa, la colección `tes_embebidos` guarda un documento por TE (239.946 documentos) con los campos de `tes` y un subdocumento con los seis perfiles. A continuación se describe cómo se midieron la carga, el espacio y el tiempo de consulta.

Para la carga y el espacio, cada estrategia se cargó completa tres veces desde cero, borrando y recreando las colecciones, con validación `$jsonSchema` en ambas para que la inserción fuera comparable. En cada carga se midieron por separado, con `time.perf_counter()`, el tiempo de cálculo de los k-mers, el de inserción y el de creación de índices. El espacio se midió con `collStats` sobre las colecciones de la última repetición: tamaño sin comprimir (`size`), tamaño en disco con la compresión de WiredTiger (`storageSize`) y tamaño de los índices (`totalIndexSize`). Las mediciones se hicieron con el equipo conectado a la corriente, las demás aplicaciones cerradas y la suspensión desactivada.

Todos los experimentos de carga, espacio y consultas se ejecutaron en el mismo equipo, descrito en la Tabla 4. Los tiempos dependen de este hardware: en un equipo con otro procesador, menos memoria o un disco más lento, los valores absolutos cambiarían, aunque se espera que la comparación entre estrategias se mantenga.

Se eligieron cuatro consultas representativas de los usos del proyecto:

- Q1, perfil de una secuencia para un k: recupera los conteos de 2-mers de una secuencia. Corresponde a la consulta obligatoria 15.
- Q2, heatmap de 2-mers por orden: calcula, mediante una agregación en MongoDB, la frecuencia promedio de cada uno de los 16 dinucleótidos para cada orden de TE. Corresponde a la figura del heatmap.
- Q3, perfil completo de una secuencia: recupera los seis perfiles (k = 1 a 6) de una secuencia. Representa el uso que favorece a la estrategia embebida, como la preparación de datos para un clasificador.
- Q4, conteo de TEs por reino y orden: agrupa y cuenta los TEs sin usar los k-mers. Representa la mayoría de las consultas obligatorias, que trabajan con los datos del TE.

Q1 y Q3 son búsquedas puntuales, por lo que cada medición repite la búsqueda para 50 identificadores elegidos al azar con semilla fija (42) entre los 239.946 de la colección `tes`; la misma muestra se usa en ambas estrategias. Q2 y Q4 recorren toda la base y se ejecutan una vez por medición. Cada consulta se ejecutó en dos variantes: con índice, dejando que MongoDB eligiera el plan, y sin índice, forzando a MongoDB a recorrer toda la colección con `hint({"$natural": 1})`, sin borrar los índices.

Para cada combinación de consulta, estrategia y variante se realizó una ejecución de calentamiento no registrada, para que los datos ya estuvieran en memoria, y luego tres repeticiones, alternando en cada una el orden de las estrategias. El tiempo se midió con `time.perf_counter()`, consumiendo todos los resultados. Además, se obtuvo una vez por combinación el plan de ejecución con `explain("executionStats")`: etapa de acceso (IXSCAN o COLLSCAN), documentos y claves examinados y tiempo de la etapa de lectura. Antes de medir, se verificó que cada consulta devolviera exactamente el mismo resultado en ambas estrategias, y que las colecciones tuvieran los documentos e índices esperados.

El procedimiento completo está en `notebooks/06_benchmark_consultas.ipyn`

4. Resultados

4.1. Consultas

Las consultas 1–9, 12–15 y las dos preguntas propias se resuelven en `04_consultas.ipynb` con `find` y `aggregate`. La Tabla 5 resume el resultado principal de cada una.

Composición.

La base tiene 239.946 TEs en 2 clases, 11 órdenes y 65 superfamilias (66 nombres, porque la de ARNt aparece escrita como `TRNA` y como `tRNA`; consulta 14). La clase I reúne el 81,2 % de los TEs, y tres órdenes concentran el 93,4 %: LTR (49,8 %), LINE (28,5 %) y TIR (15,2 %). Las superfamilias más abundantes son Copia (21,2 %) y Gypsy (18,3 %). Cada registro es una entrada de biblioteca, normalmente el consenso de una familia (una secuencia que la representa), así que los conteos miden familias anotadas y no la fracción del genoma que ocupan. Por reino, las plantas están dominadas por LTR (72,3 %) y los animales por LINE (44,0 %) (consulta 3). El tamaño de las bibliotecas varía mucho: *Malus domestica* tiene 1.627 TEs, mientras que 326 especies (12 %) tienen uno solo (consulta 4), lo que limita las comparaciones entre especies.

Longitud y GC.

Los TEs de clase I miden en promedio más del doble que los de clase II (4.453 frente a 2.137 nt; consulta 5). En TIR y ERV el promedio supera ampliamente a la mediana, señal de distribuciones con cola larga. Varios órdenes tienen límites exactos (LINE entre 500 y 6.000 nt; SINE y MITE hasta 500 nt), y ningún TE mide menos de 100 nt salvo algunos SINE (mínimo 81 nt; consulta 5). En plantas y hongos hay TEs de exactamente 100 nt incluso en LTR y Helitron (consulta 7), lo que sugiere umbrales del pipeline de origen. El GC promedio es mayor en hongos (46,5 %) que en animales (44,6 %) y plantas (41,4 %), y entre los órdenes destacan SINE (53,1 %) y DIRS (52,9 %) (consulta 6). El TE más largo de la base es un Gypsy de 19.926 nt de *Helianthus annuus*.

Búsquedas.

Las consultas 8, 9 y 15 usan índices que examinan exactamente los documentos devueltos: el índice único `{pdb_id}` (1 clave), el compuesto `{species_name, classification.te_superfamily}` (2 claves para los 2 Helitron de *H. mephisto*) y `{pdb_id, k}` (1 documento). La consulta 9 muestra por qué hace falta normalizar los nombres: con el nombre del CSV, *Beta vulgaris subsp vulgaris*, sus 84 TEs de L1 solo aparecen después de aplicar `normalize_species`.

Relaciones y calidad.

En la consulta 12, unir los 239.946 TEs con su especie mediante `$lookup` tardó 45,6 s, frente a 1,1 s al agrupar primero por especie (2.790 búsquedas en lugar de 239.946): conviene reducir antes de unir. Por origen (consulta 13), Dfam aporta 386 especies y 31.592 TEs, y RM2+MCH 2.404 especies y 208.354 TEs. El promedio de TEs por especie es parecido (81,8 y 86,7), pero los TEs de RM2+MCH son mucho más largos (4.376 frente a 1.654 nt), en parte porque Dfam tiene más TIR y SINE. Como 385 de las 386 especies de Dfam son animales, esta comparación queda mezclada con la del reino. El control de calidad (consulta 14) encontró 10.319 TEs (4,3 %) clasificados solo hasta orden, sobre todo en TIR (16,4 % del orden) y SINE (21,2 %); la superfamilia de ARNt escrita como `TRNA` y como `tRNA`; 4 TEs `SINE/ERV1`, y 6 TEs ERV en hongos (consulta 7). También confirmó que no hay TEs sin especie y que cada TE tiene sus 6 perfiles de k-mers.

Preguntas propias.

La propia 1 mide la diversidad de superfamilias por especie. Está fuertemente correlacionada con el número de TEs (ρ de Spearman = 0,92; $n = 2.790$): las especies con más TEs casi siempre muestran más superfamilias, así que las diferencias entre grupos reflejan sobre todo el tamaño de las bibliotecas. La propia 2 calcula la razón CpG observado/esperado con los perfiles de $k = 1$ y $k = 2$, que compara cuántos `CG` tiene cada TE con los que tendría por azar según su cantidad de C y de G: un valor de 1 es lo esperado y uno menor indica que faltan `CG`. Los TEs de plantas (0,62) y animales (0,66) están empobrecidos en CpG, mucho más que los de hongos (0,86). Esto concuerda con la desaminación de las citosinas metiladas, que con el tiempo convierte los CG en TG o CA (Bird, 1980), y con que la metilación del ADN es escasa o ausente en varios hongos (Zemach et al., 2010). Fue la consulta más costosa (211 s), porque necesita un `$lookup` a `tes` para conocer el reino de cada TE.

Bibliotecas (consultas 10 y 11).

Las consultas 10 y 11 generan bibliotecas `.fasta` a partir de la base, sin leer el FASTA original. Una misma función recibe el filtro como parámetro (`species_name` para la especie y `kingdom` para el reino), consulta `tes` trayendo solo los campos del encabezado y recupera las secuencias de `secuencias` por lotes de 5.000 `_id` con `$in`. Cada registro se escribe con el encabezado original, `>ID#CLASE/ORDEN/SUPERFAMILIA @Especie`, y la secuencia sin modificar. Si un archivo supera 100 MB, se comprime con gzip. La función comprueba que el número de secuencias escritas coincida con `count_documents` sobre el mismo filtro.

Para la especie se usó *Halicephalobus mephisto*: su biblioteca tiene 10 secuencias (7 TIR, 2 HELITRON y 1 LINE) y ocupa 17 KB. Para el reino se usó Fungi: 19.797 secuencias y 69,8 MB, por debajo del límite, así que no fue necesario comprimirla. Las bibliotecas de Animalia (101.144 TEs) y Plantae (119.005) se generan con la misma función cambiando el parámetro.

4.2. Visualización de los datos

Las cuatro figuras se generan en `05_bibliotecas_y_figuras.ipynb` a partir de consultas a la base, con matplotlib (Hunter, 2007) y seaborn (Waskom, 2021).

La composición por orden cambia entre reinos (Figura 2). Se usa el porcentaje dentro de cada reino porque Fungi tiene seis veces menos TEs que Plantae. En Plantae dominan los LTR (72,3 %). En Animalia el orden más frecuente es LINE (44,0 %), seguido de LTR (25,1 %) y TIR (21,4 %). Fungi queda en un punto intermedio, con LTR (40,1 %) y LINE (31,0 %), y es el reino con más HELITRON (8,6 %) y DIRS (3,8 %). SINE y MITE solo superan el 1 % en Animalia, y PLE, RETROPOSON, CRYPTON y MAVERICK no llegan al 0,5 % en ningún reino.

La longitud separa dos grupos de órdenes (Figura 3). SINE, CRYPTON y MITE son cortos, con medianas entre 254 y 345 nt. LTR, DIRS y MAVERICK son los más largos, con medianas por encima de 5.000 nt, y en LTR está el TE más largo de la base (19.926 nt). HELITRON y TIR tienen la mayor dispersión, con valores desde 100 nt hasta 14.819 y 9.991 nt. SINE y MITE nunca pasan de 500 nt y LINE nunca baja de esa cifra; estos cortes parecen venir de los criterios de la biblioteca de origen, lo que queda por verificar en la documentación de PanTEon.

El contenido GC es mayor en Fungi (mediana 47,1 %) que en Animalia (44,1 %) y Plantae (41,0 %), aunque las distribuciones se solapan mucho (Figura 4). Plantae tiene además la menor dispersión (desviación estándar de 6,5 puntos, frente a 8,1 y 9,0).

En los 2-mers (Figura 5), CG es el menos frecuente en seis de los 11 órdenes (LINE, LTR, TIR, MITE, HELITRON y RETROPOSON) y alcanza su mínimo en LINE y LTR (0,030), lo que concuerda con el déficit de CpG que deja la metilación del ADN (Bird, 1980). LINE y LTR tienen perfiles casi iguales. TIR, MITE, HELITRON y MAVERICK forman otro grupo, rico en AA y TT (frecuencias de 0,096 a 0,106). SINE, RETROPOSON y DIRS son los más distintos: más CC y GG y menos AT, TA o TT. Los promedios de CRYPTON, MAVERICK y RETROPOSON se basan en menos de 120 TEs cada uno y deben leerse con cautela.

4.3. Evaluación del diseño

Carga y espacio.

La Tabla 6 y la Figura 6 resumen la carga y el espacio. El cálculo de los k-mers domina la carga, con unos 125 s, y es igual en ambas estrategias. La embebida carga un 5,5 % más rápido, sobre todo porque crea menos índices: 8,9 de los 10,9 s de diferencia. En disco, ambas ocupan prácticamente lo mismo; repetir `te_id`, `pdb_id`, `te_order` y k en 1,44 millones de documentos cuesta poco después de la compresión. La diferencia está en los índices, unas 11 veces más grandes en `kmers` porque indexan seis veces más documentos, aunque siguen siendo menos del 2 % del total. Las tres repeticiones fueron estables, con una desviación de 2 s o menos en el total.

Consultas.

La Tabla 7 resume los tiempos de las cuatro consultas, la Tabla 8 los planes de ejecución obtenidos con `explain` y la Figura 7 compara los tiempos en escala logarítmica.

Tabla 5 Resumen de las consultas obligatorias y su resultado principal.

N.°	Pregunta	Resultado clave
1	TEs por clase, orden y superfamilia	Clase I 81,2 %; LTR 49,8 %, LINE 28,5 %, TIR 15,2 %
2	TEs por especie, por clase, orden y superfamilia	18.513 combinaciones especie–superfamilia (6,6 por especie)
3	TEs por clase y orden entre reinos y entre phyla	Plantas: LTR 72,3 %; animales: LINE 44,0 %
4	10 especies con más y con menos TEs	Máximo: *M. domestica* (1.627); 326 especies con 1 TE
5	Longitud promedio, mínima y máxima por clase, orden y superfamilia	Clase I 4.453 nt, clase II 2.137 nt
6	GC promedio por orden y por reino	Hongos 46,5 %, animales 44,6 %, plantas 41,4 %
7	TEs más largos y más cortos de cada reino	Gypsy de 19.926 nt (*H. annuus*); SINE de 81 nt
8	Búsqueda de una secuencia por su nombre	`PDB00000001` con su taxonomía; IXSCAN, 1 clave
9	Secuencias de una superfamilia en una especie	*H. mephisto*: 2 HELITRON; 2 claves examinadas
10	Biblioteca `.fasta` de una especie	*H. mephisto*: 10 secuencias, 17 KB
11	Biblioteca `.fasta` de un reino	Fungi: 19.797 secuencias, 69,8 MB (sin comprimir)
12	`$lookup` de secuencias con la taxonomía de su especie	45,6 s directa frente a 1,1 s agrupando antes
13	Comparación por origen (Dfam frente a RM2+MCH)	Dfam 386 especies, 1.654 nt; RM2+MCH 2.404, 4.376 nt
14	Control de calidad	10.319 TEs (4,3 %) solo hasta orden; 0 TEs sin especie
15	Perfil de k-mers de una secuencia para un k	`PDB00000001`, $k = 2$: 1.990 ventanas; 1 documento
P1	Superfamilias por especie según reino, fuente y número de TEs	$\rho = 0{,}92$ con el número de TEs
P2	Empobrecimiento de CpG por orden y reino	CpG o/e: plantas 0,62; animales 0,66; hongos 0,86

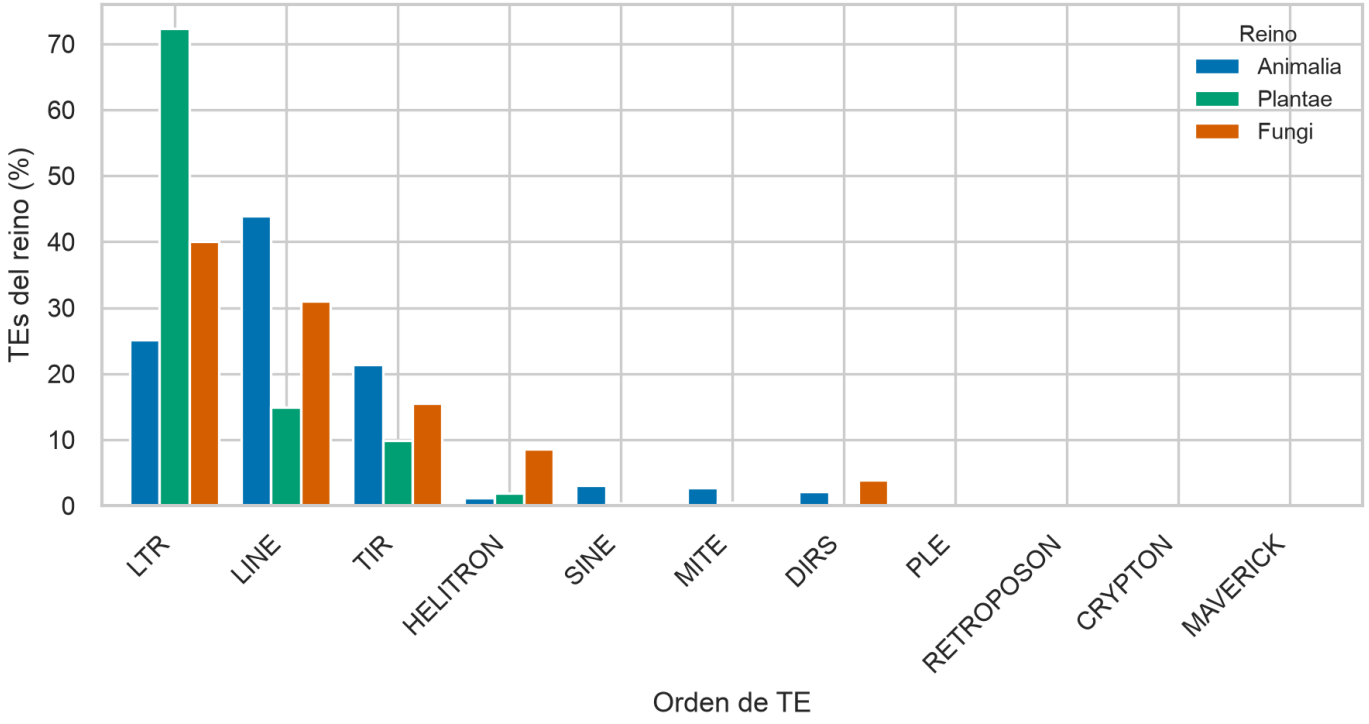


Figura 2 Composición de TEs por reino: porcentaje de TEs de cada orden dentro de Animalia (101.144 TEs), Plantae (119.005) y Fungi (19.797).

Texto alternativo: Gráfico de barras agrupadas con los 11 órdenes de TE en el eje horizontal y el porcentaje de TEs del reino en el vertical, con una barra por reino. La barra más alta es LTR en Plantae, con un 72 %. En Animalia la barra más alta es LINE, con un 44 %. Desde SINE hacia la derecha, todas las barras están por debajo del 4 %.

Búsquedas por secuencia (Q1 y Q3).

Con índice, ambas estrategias acceden directamente a los documentos de la secuencia (IXSCAN) y obtienen tiempos equivalentes: 0,114 frente a 0,115 ms por búsqueda en Q1, y 0,372 frente a 0,318 ms en Q3. En Q3, la estrategia embebida lee un documento en lugar de seis, pero la diferencia es menor que la variación entre repeticiones. Sin índice, MongoDB recorre la colección en cada búsqueda y los tiempos aumentan a cientos de milisegundos; en este caso la embebida es más rápida (360 frente a 596 ms en Q1, y 267 frente a 540 ms en Q3), porque tiene seis veces menos documentos y, en Q3, detiene el recorrido al encontrar el documento. En ambas consultas, el índice influye mucho más que la estrategia de almacenamiento: acelera las búsquedas entre unas 840 y 5.000 veces.

Heatmap de 2-mers (Q2).

Las cuatro combinaciones realizan el mismo cálculo y solo difieren en la lectura: la colección con índice lee únicamente los 239.946 documentos de k = 2 mediante el índice `{k, te_order}`, mientras que la embebida, que no puede filtrar por k, lee los 239.946 documentos completos. Aun así, el tiempo total fue similar en todos los casos

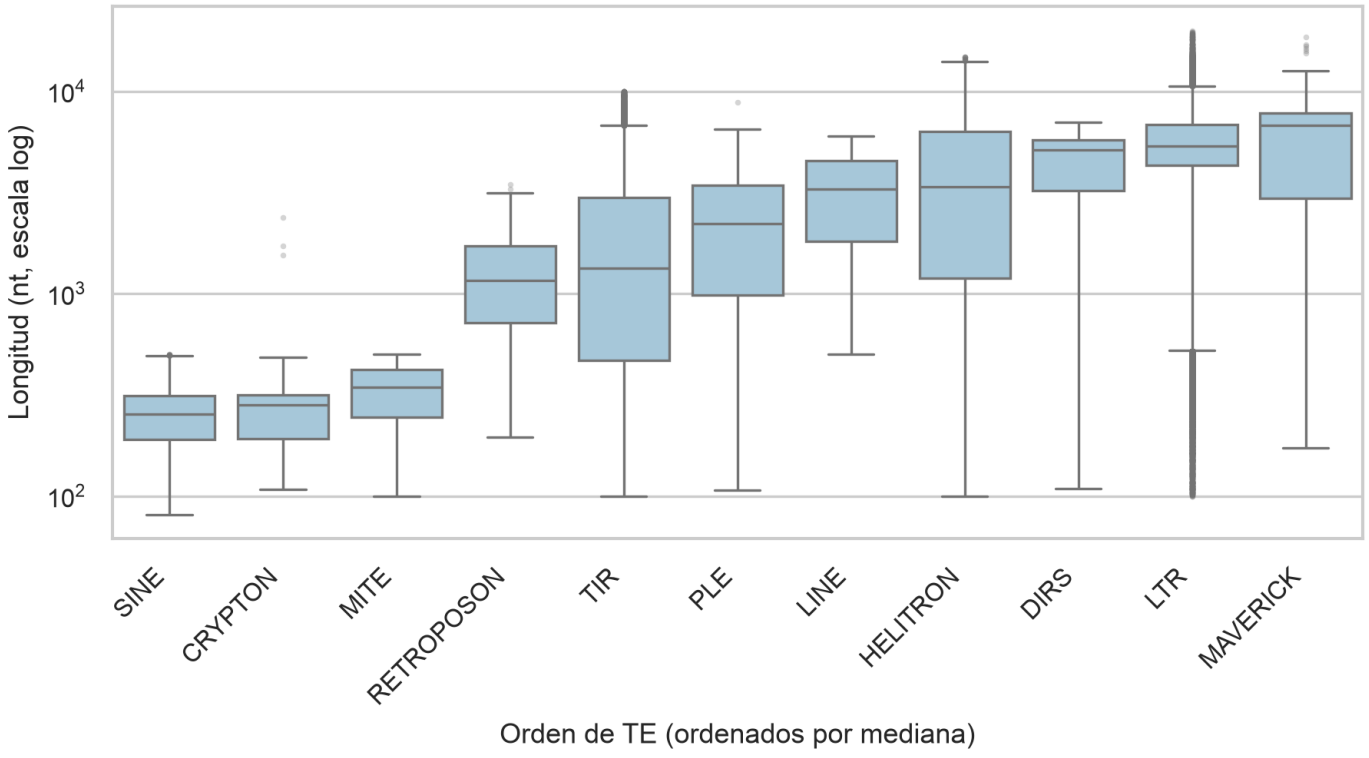


Figura 3 Distribución de la longitud de los TEs por orden, en escala logarítmica. Los órdenes están ordenados por su mediana.

Texto alternativo: Diagrama de cajas con un orden de TE por caja y la longitud en nucleótidos en escala logarítmica. SINE, CRYPTON y MITE, a la izquierda, tienen cajas por debajo de 500 nucleótidos. Las medianas suben hacia la derecha hasta LTR y MAVERICK, por encima de 5.000. LTR tiene muchos valores atípicos en ambos extremos.

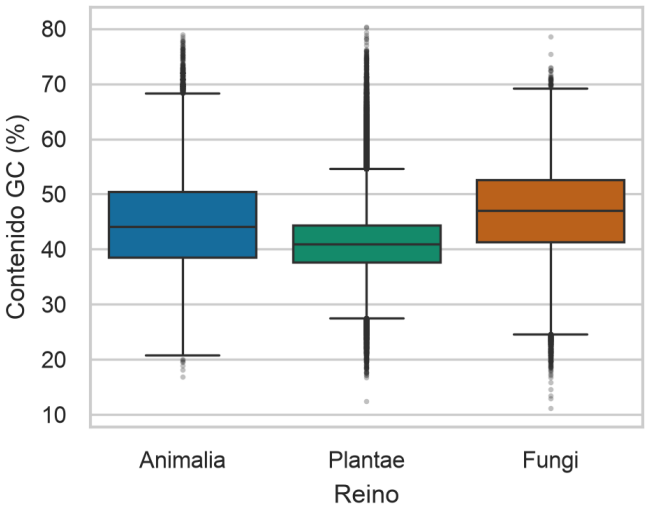


Figura 4 Contenido GC de los TEs por reino, en porcentaje.

Texto alternativo: Diagrama de cajas con tres cajas, una por reino, y el contenido GC en porcentaje en el eje vertical. Las medianas son de alrededor del 44 % en Animalia, del 41 % en Plantae y del 47 % en Fungi. Las tres cajas se solapan, y la de Plantae es la más estrecha.

(entre 11,3 y 12,4 s en promedio), con una variación alta entre repeticiones (de 3,5 a 16,7 s en una misma combinación). Con los datos

Tabla 6 Carga y espacio por estrategia (media ± desviación estándar, $n = 3$; espacio en MB de la última repetición).

Medida	Colección `kmers`	Embebida
Documentos	1.439.676	239.946
Cálculo (s)	125,5 ± 2,1	125,8 ± 0,4
Inserción (s)	50,2 ± 0,2	47,9 ± 0,1
Índices (s)	22,0 ± 0,4	13,1 ± 1,5
Carga total (s)	197,7 ± 2,0	186,8 ± 1,8
Datos en disco (MB)	3.030,6	3.037,1
Índices (MB)	57,7	5,2

en memoria, el costo dominante es el cálculo de la agregación (convertir conteos en frecuencias, separar con `$unwind` unos 3,8 millones de elementos y sumarlos por orden), que es idéntico en ambas estrategias. La diferencia aparece en la etapa de lectura: según `explain`, la colección con índice leyó los datos en 4,0 s, frente a 15 a 17 s en las demás combinaciones, cifra que se toma como un indicio por provenir de una sola ejecución previa al calentamiento.

Consulta sin k-mers (Q4).

En Q4 ningún índice es aplicable y las cuatro combinaciones recorren los 239.946 documentos. La colección `tes` resolvió la consulta en 128 a 138 ms, mientras que `tes_embebidos` tardó entre 352 y 581 ms, es decir, entre 2,5 y 4,5 veces más. La diferencia se debe al tamaño de los documentos: unos 34.000 bytes en promedio en la estrategia embebida, frente a unos 300 en `tes`, que deben leerse completos aunque la consulta no use los k-mers.

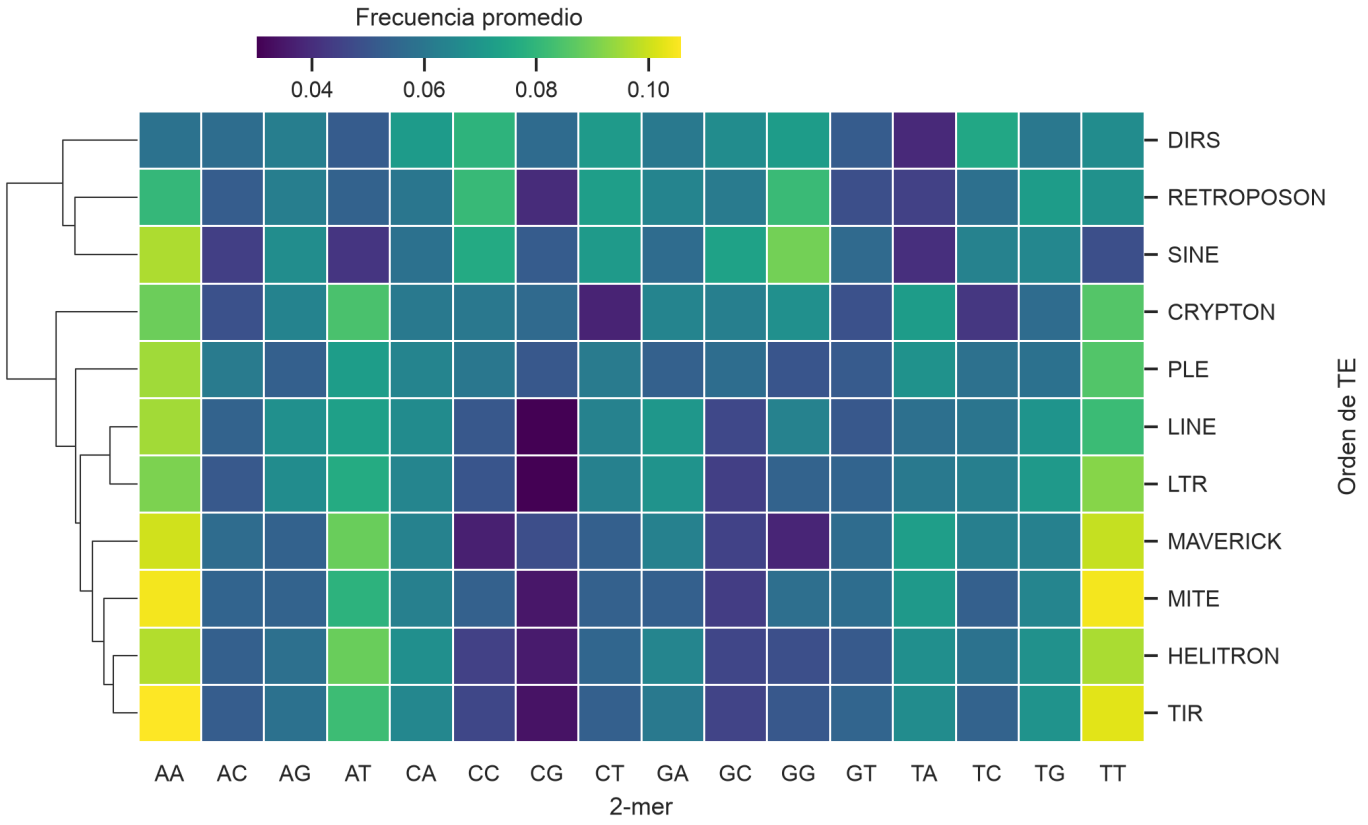


Figura 5 Frecuencia promedio de los 16 2-mers por orden de TE. Las filas se agrupan por similitud del perfil (dendrograma a la izquierda).

Texto alternativo: Mapa de calor con los 11 órdenes de TE en las filas y los 16 2-mers en las columnas; el color va de morado, para frecuencias bajas, a amarillo, para altas. La columna CG es la más oscura en LINE, LTR, TIR, MITE, HELITRON y RETROPOSON. Las columnas AA y TT son claras en TIR, MITE, HELITRON y MAVERICK. Un dendrograma separa arriba a DIRS, RETROPOSON y SINE del resto.

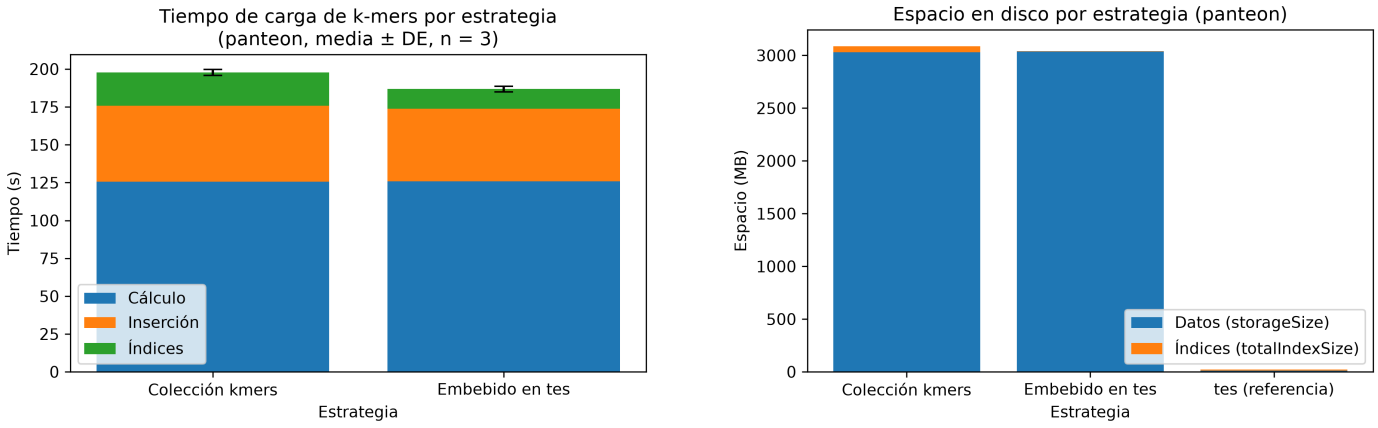


Figura 6 Carga y espacio de las dos estrategias de k-mers. Izquierda: tiempo de carga separado en cálculo, inserción e índices (media ± desviación estándar del total, $n = 3$). Derecha: espacio en disco de los datos comprimidos (`storageSize`) y de los índices (`totalIndexSize`); la colección `tes` se muestra como referencia.

Texto alternativo: Dos gráficos de barras apiladas. En el de carga, la colección `kmers` y la estrategia embebida tienen unos 125 s de cálculo y unos 50 s de inserción; la parte de índices es más alta en `kmers`, con un total de unos 198 s frente a unos 187 s. En el de espacio, ambas estrategias ocupan unos 3.000 MB de datos; la franja de índices solo se ve en `kmers`, y la barra de `tes` es casi invisible a esa escala.

5. Discusión

Ventajas y limitaciones según las consultas.

Las consultas respaldan las decisiones del modelo. Copiar `species_name`, `kingdom`, `phylum` y `source` en `tes` permitió resolver

Tabla 7 Tiempo de consulta por estrategia y variante (media ± desviación estándar, $n = 3$). Q1 y Q3: milisegundos por búsqueda; Q2 y Q4: tiempo total.

Consulta	Colección, con índice	Colección, sin índice	Embebida, con índice	Embebida, sin índice
Q1 (ms por búsqueda)	0,114 ± 0,004	596 ± 257	0,115 ± 0,003	360 ± 257
Q2 (s)	11,7 ± 7,1	12,0 ± 4,8	11,3 ± 5,5	12,4 ± 4,8
Q3 (ms por búsqueda)	0,372 ± 0,161	540 ± 228	0,318 ± 0,072	267 ± 193
Q4 (ms)	138 ± 13	128 ± 1	352 ± 189	581 ± 188

Tabla 8 Plan de ejecución según `explain("executionStats")` (una ejecución por combinación). En Q1 y Q3 corresponde a la búsqueda de una secuencia; en Q2 y Q4, a la etapa de lectura de la agregación.

Consulta y estrategia	Con índice: etapa / documentos / ms	Sin índice: etapa / documentos / ms
Q1, colección	IXSCAN / 1 / 0	COLLSCAN / 1.439.676 / 12.381
Q1, embebida	IXSCAN / 1 / 0	COLLSCAN / 239.946 / 12.730
Q2, colección	IXSCAN / 239.946 / 4.019	COLLSCAN / 1.439.676 / 16.031
Q2, embebida	COLLSCAN / 239.946 / 16.879	COLLSCAN / 239.946 / 15.276
Q3, colección	IXSCAN / 6 / 0	COLLSCAN / 1.439.676 / 9.819
Q3, embebida	IXSCAN / 1 / 0	COLLSCAN / 174.247 / 10.640
Q4, colección	COLLSCAN / 239.946 / 170	COLLSCAN / 239.946 / 176
Q4, embebida	COLLSCAN / 239.946 / 2.428	COLLSCAN / 239.946 / 665

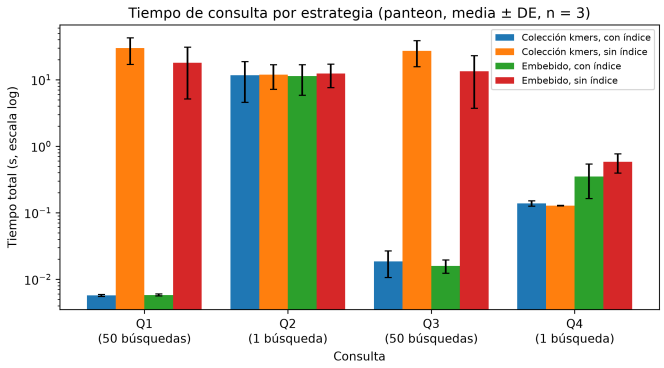


Figura 7 Tiempo total de consulta por estrategia y variante (media ± desviación estándar, $n = 3$, escala logarítmica). Q1 y Q3 incluyen las 50 búsquedas.

Texto alternativo: Gráfico de barras agrupadas con las consultas Q1 a Q4 en el eje horizontal y el tiempo total en segundos, en escala logarítmica, en el vertical; cada consulta tiene cuatro barras: colección con y sin índice y embebida con y sin índice. En Q1 y Q3 las barras sin índice son unas mil veces más altas que las barras con índice. En Q2 las cuatro barras son parecidas, de unos 12 s. En Q4 las barras de la estrategia embebida son más altas que las de la colección.

sin `$lookup` las consultas 1–3, 5–7, 9 y 13, y cuando la unión fue necesaria, agrupar antes de unir la hizo unas 42 veces más rápida (consulta 12). Los índices sobre `pdb_id`, sobre especie y superfamilia y sobre `pdb_id` y k hicieron que las búsquedas 8, 9 y 15 leyeran solo los documentos devueltos. La principal limitación apareció en la propia 2: como `kmers` solo copia `te_order`, conocer el reino exige un `$lookup` por TE, y la consulta tardó 211 s. Si ese uso fuera frecuente, convendría copiar también `kingdom` en `kmers`. Guardar `species_name` tal como viene en el FASTA mantiene idénticos los encabezados de las bibliotecas, pero obliga a normalizar el nombre para filtrar en 4 especies (consulta 9).

Estrategia de almacenamiento según el tipo de uso.

Los resultados muestran que la conveniencia de cada estrategia depende del tipo de consulta. En las búsquedas de k-mers por secuencia, ambas son equivalentes cuando existe el índice adecuado, por lo que lo determinante es indexar los campos de búsqueda. En el heatmap empatan con el volumen actual, aunque la colección aparte lee menos datos. La diferencia clara aparece en las consultas que no usan k-mers, donde la estrategia embebida es entre 2,5 y 4,5 veces más lenta porque arrastra los perfiles en cada documento. Como la mayoría de las consultas del proyecto (las consultas 1 a 14) no usan k-mers, se mantiene la colección aparte. La alternativa embebida solo ofrece una carga un 5,5 % más rápida (186,8 ± 1,8 s frente a 197,7 ± 2,0 s) e índices unas 11 veces más pequeños (5,2 frente a 57,7 MB), con un espacio de datos prácticamente igual (3.037,1 frente a 3.030,6 MB), ventajas que no compensan la penalización en el resto de consultas.

Crecimiento de los datos.

Con el volumen actual, cada estrategia ocupa unos 8 GB sin comprimir y buena parte de los datos cabe en la memoria del equipo de pruebas. Si los datos crecieran 10 o 100 veces, dejarían de caber en memoria y la lectura desde disco pasaría a dominar el tiempo de las consultas que recorren toda la base. En ese escenario, es esperable que la colección aparte con índice, cuya etapa de lectura fue unas 4 veces más rápida en el heatmap, supere a la embebida, y que la penalización de la embebida en consultas sin k-mers aumente. Esta hipótesis no se midió en este trabajo.

Limitaciones del experimento.

- Variación por la caché. En Q2 y en las variantes sin índice, los tiempos variaron mucho entre repeticiones (en Q2, entre 3,5 y 16,7 s en una misma combinación). Al alternar dos colecciones de unos 8 GB sin comprimir, una desplaza a la otra de la memoria, y cada medición depende de si los datos ya estaban cargados.
- Pocas repeticiones. Tres repeticiones son pocas para conclusiones firmes cuando la variación es tan alta.
- Un solo equipo. Todas las mediciones se hicieron en una misma máquina (Apple M5, 24 GB); en equipos con menos memoria el efecto de la caché sería mayor.
- `explain` en frío. Los tiempos de lectura de `explain` provienen de una sola ejecución previa al calentamiento, por lo que solo se usan como indicio.
- Formato no comparado. El benchmark compara la ubicación de los k-mers (colección aparte frente a embebidos), no su formato; el ahorro del diccionario disperso frente a un arreglo denso no se midió.

Amenazas a la validez de los resultados biológicos.

Los registros son consensos de familias y no copias en el genoma, así que los conteos miden familias anotadas. El tamaño de las bibliotecas

varía mucho (326 especies tienen un solo TE) y determina la diversidad observada (propia 1). Las dos fuentes no son comparables en lo taxonómico, porque casi todas las especies de Dfam son animales (consulta 13). Además, hay indicios de cortes artificiales de longitud y de errores de clasificación en las bibliotecas de origen (consultas 5, 7 y 14), que se documentaron pero no se corrigieron.

6. Conclusiones

Se diseñó e implementó en MongoDB un modelo de datos que integra las 239.946 secuencias de elementos transponibles de PanTEon Database v1.6.1, la taxonomía de sus 2.790 especies y los perfiles de k-mers de k = 1 a 6 de cada secuencia. El modelo organiza la información en cuatro colecciones: `especies`, `tes`, `secuencias` y `kmers`. Combina referencias, para mantener la taxonomía en un solo lugar y permitir uniones con `$lookup`, con la copia de los campos más consultados, para resolver sin uniones las consultas por especie y por reino.

La exploración de los datos mostró que las fuentes eran consistentes en lo esencial, pero con particularidades que había que documentar: identificadores con formatos distintos según la biblioteca de origen, superfamilias que repiten el nombre del orden, secuencias mayoritariamente en minúsculas, presencia de N y nombres de especie que solo diferían en la puntuación. Todas se resolvieron sin modificar los datos originales.

La evaluación experimental mostró que guardar los k-mers en una colección aparte no tiene costo apreciable en carga ni en espacio frente a embeberlos, y que, con los índices adecuados, ambas estrategias son equivalentes en las búsquedas por secuencia. La colección aparte evita, en cambio, que las consultas que no usan k-mers, la mayoría en este proyecto, carguen con el peso de los perfiles, lo que las hace entre 2,5 y 4,5 veces más rápidas. Como trabajo futuro, queda evaluar el comportamiento con volúmenes que no quepan en memoria y comparar formatos de almacenamiento de los perfiles.

Las consultas mostraron que el modelo responde todas las preguntas planteadas, desde los conteos globales hasta la generación de bibliotecas `.fasta` por especie y por reino. Los campos copiados y los índices permiten resolver la mayoría sin uniones y leyendo solo los documentos necesarios. Los datos muestran diferencias claras entre reinos: las plantas están dominadas por LTR, los animales por LINE, y los hongos tienen más GC y menos empobrecimiento de CpG. Estas comparaciones, sin embargo, dependen del tamaño y del origen de cada biblioteca.

7. Disponibilidad de datos y código

El código, los notebooks, los resultados de los experimentos y la documentación del modelo están disponibles en el repositorio del proyecto: `https://github.com/ThomasToro/transposable-elements-db`. Los datos de entrada son los archivos `PanTEon_Database_v1.6.1.fasta` y `PanTEon_Database_metadata_v1.6.1.csv` de PanTEon Database v1.6.1, disponibles en Zenodo (`https://zenodo.org/records/19372616`) bajo licencia CC BY 4.0 (Orozco-Arias et al., 2026b).

Para reproducir los resultados, el archivo `README.md` describe la instalación del entorno (`environment.yml`, con Python 3.11 y MongoDB 7.0), la descarga de los datos y el orden de ejecución de los notebooks. La base completa se reconstruye ejecutando en orden los notebooks 01, 02 y 03, o restaurando los respaldos de MongoDB descritos en `resultados/dump_info.md`. Las funciones del pipeline cuentan con pruebas automatizadas (`python -m pytest tests/`), y los benchmarks se encuentran en los notebooks `06_benchmark_carga_espacio.ipynb` y `06_benchmark_consultas.ipynb`.

8. Contribuciones de los autores

T.A.T.H.: conceptualización, metodología, curación de datos (FASTA), software, investigación (benchmark de consultas), visualización (modelo de datos y benchmark de consultas), administración del proyecto, redacción (borrador original). J.S.A.R.: curación de datos (CSV y cruce), software (pipeline de carga y k-mers), investigación (benchmark de carga y espacio), visualización (benchmark de carga y espacio), redacción (borrador original). J.E.H.M.: software (consultas), análisis formal (consultas), redacción (borrador original). I.Q.M.: software (bibliotecas), visualización (figuras de los datos), redacción (borrador original, revisión y edición).

9. Financiación

Este trabajo no recibió financiación.

10. Conflicto de intereses

Los autores declaran no tener conflicto de intereses.

11. Agradecimientos

Al docente Simón Gaviria Orrego, por la orientación durante el curso de Bases de Datos II.

Referencias

W. Bao, K. K. Kojima, and O. Kohany. Repbase Update, a database of repetitive elements in eukaryotic genomes. *Mobile DNA*, 6(1):11, 2015. doi: 10.1186/s13100-015-0041-9.

A. P. Bird. DNA methylation and the frequency of CpG in animal DNA. *Nucleic Acids Research*, 8(7):1499–1504, 1980. doi: 10.1093/nar/8.7.1499.

P. J. A. Cock, T. Antao, J. T. Chang, B. A. Chapman, C. J. Cox, A. Dalke, I. Friedberg, T. Hamelryck, F. Kauff, B. Wilczynski, and M. J. L. de Hoon. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009. doi: 10.1093/bioinformatics/btp163.

J. M. Flynn, R. Hubley, C. Goubert, J. Rosen, A. G. Clark, C. Feschotte, and A. F. Smit. RepeatModeler2 for automated genomic discovery of transposable element families. *Proceedings of the National Academy of Sciences*, 117(17):9451–9457, 2020. doi: 10.1073/pnas.1921046117.

C. R. Harris, K. J. Millman, S. J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

J. D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.

W. McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

MongoDB, Inc. MongoDB 7.0 manual. `https://www.mongodb.com/docs/v7.0/`, 2026a. Consultado el 29 de octubre de 2026.

MongoDB, Inc. PyMongo 4.19 documentation. `https://pymongo.readthedocs.io/`, 2026b. Consultado el 29 de octubre de 2026.

S. Orozco-Arias, P. Sierra, R. Durbin, and J. González. MCHelper automatically curates transposable element libraries across eukaryotic species. *Genome Research*, 34(12):2256–2268, 2024. doi: 10.1101/gr.278821.123.

S. Orozco-Arias, I. Ferrer-Pomer, F. Rodrigues de Góes, S. Gaviria-Orrego, J. Gómiz-Fernández, J. Llatser-Torres, A. R. Paschoal, R. Guyot, and T. Gabaldón. PanTEon: a cross-kingdom framework to guide the design of transposable element classifiers. *bioRxiv*, 2026a. doi: 10.64898/2026.04.01.715927. Preprint.

S. Orozco-Arias, I. Ferrer-Pomer, F. Rodrigues de Góes, S. Gaviria-Orrego, J. Gómiz-Fernández, J. Llatser-Torres, A. R. Paschoal, R. Guyot, and T. Gabaldón. [dataset] PanTEon Database: A cross-kingdom, automatically curated reference for transposable elements, version 1.6.1. Zenodo, 2026b. doi: 10.5281/zenodo.19372616.

J. Storer, R. Hubley, J. Rosen, T. J. Wheeler, and A. F. Smit. The Dfam community resource of transposable element families, sequence models, and genome annotations. *Mobile DNA*, 12(1):2, 2021. doi: 10.1186/s13100-020-00230-y.

M. L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021. doi: 10.21105/joss.03021.

T. Wicker, F. Sabot, A. Hua-Van, J. L. Bennetzen, P. Capy, B. Chalhoub, A. Flavell, P. Leroy, M. Morgante, O. Panaud, E. Paux, P. SanMiguel, and A. H. Schulman. A unified classification system for eukaryotic transposable elements. *Nature Reviews Genetics*, 8(12):973–982, 2007. doi: 10.1038/nrg2165.

A. Zemach, I. E. McDaniel, P. Silva, and D. Zilberman. Genome-wide evolutionary analysis of eukaryotic DNA methylation. *Science*, 328 (5980):916–919, 2010. doi: 10.1126/science.1186366.